

DevOps Lab

Documentation

Program 8:

Create Kubernetes Pods using YAML Descriptors

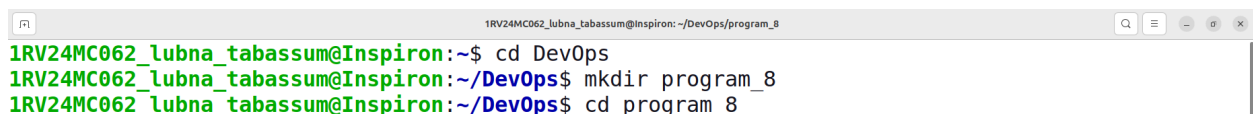
The program demonstrates how to create and manage Kubernetes Pods using declarative YAML descriptor files instead of imperative commands.

Project Structure:

```
program_8/  
—>nodejs-configmap.yaml  
—>nodejs-pod.yaml  
—>server.js
```

Procedure:

Create a folder (ex: program_8)

A terminal window screenshot showing the creation of a directory and navigation. The terminal title is '1RV24MC062_lubna_tabassum@Inspiron: ~/DevOps/program_8'. The commands and their outputs are: 'cd DevOps', 'mkdir program_8', and 'cd program_8'.

```
1RV24MC062_lubna_tabassum@Inspiron:~$ cd DevOps  
1RV24MC062_lubna_tabassum@Inspiron:~/DevOps$ mkdir program_8  
1RV24MC062_lubna_tabassum@Inspiron:~/DevOps$ cd program_8
```

Step 1: Start Minikube

minikube start

Check the status of Minikube

minikube status

```

1RV24MC062_lubna_tabassum@Inspiron:~/DevOps/program_8$ minikube start
🐳 minikube v1.37.0 on Ubuntu 24.04
🌟 Using the docker driver based on existing profile
👉 Starting "minikube" primary control-plane node in "minikube" cluster
📦 Pulling base image v0.0.48 ...
🔄 Restarting existing docker container for "minikube" ...

🚨 Docker is nearly out of disk space, which may cause deployments to fail! (90% of capacity).
You can pass '--force' to skip this check.
💡 Suggestion:

Try one or more of the following to free up space on the device:

1. Run "docker system prune" to remove unused Docker data (optionally with "-a")
2. Increase the storage allocated to Docker for Desktop by clicking on:
   Docker icon > Preferences > Resources > Disk Image Size
3. Run "minikube ssh -- docker system prune" if using the Docker container runtime
🍷 Related issue: https://github.com/kubernetes/minikube/issues/9024

🔧 Preparing Kubernetes v1.34.0 on Docker 28.4.0 ...
🔍 Verifying Kubernetes components...
   ■ Using image registry.k8s.io/metrics-server/metrics-server:v0.8.0
   ■ Using image docker.io/kubernetesui/dashboard:v2.7.0
   ■ Using image gcr.io/k8s-minikube/storage-provisioner:v5
   ■ Using image docker.io/kubernetesui/metrics-scraper:v1.0.8
💡 Some dashboard features require the metrics-server addon. To enable all features please run:

1RV24MC062_lubna_tabassum@Inspiron:~/DevOps/program_8$ minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured

```

Step 2: Create nodejs-configmap.yaml file with the following code

nano nodejs-configmap.yaml

```

GNU nano 7.2 nodejs-configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: nodejs-app-config
data:
  server.js: |
    const http = require("http");
    const port = 3000;

    const server = http.createServer((req, res) => {
      res.end("Hello from Node.js running inside Kubernetes!");
    });

    server.listen(port, () => {
      console.log(`Server running at port ${port}`);
    });

```

[Read 16 lines]

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location
 ^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line

Step 3: Create `nodejs-pod.yaml` file with the following code
`nano nodejs-pod.yaml`

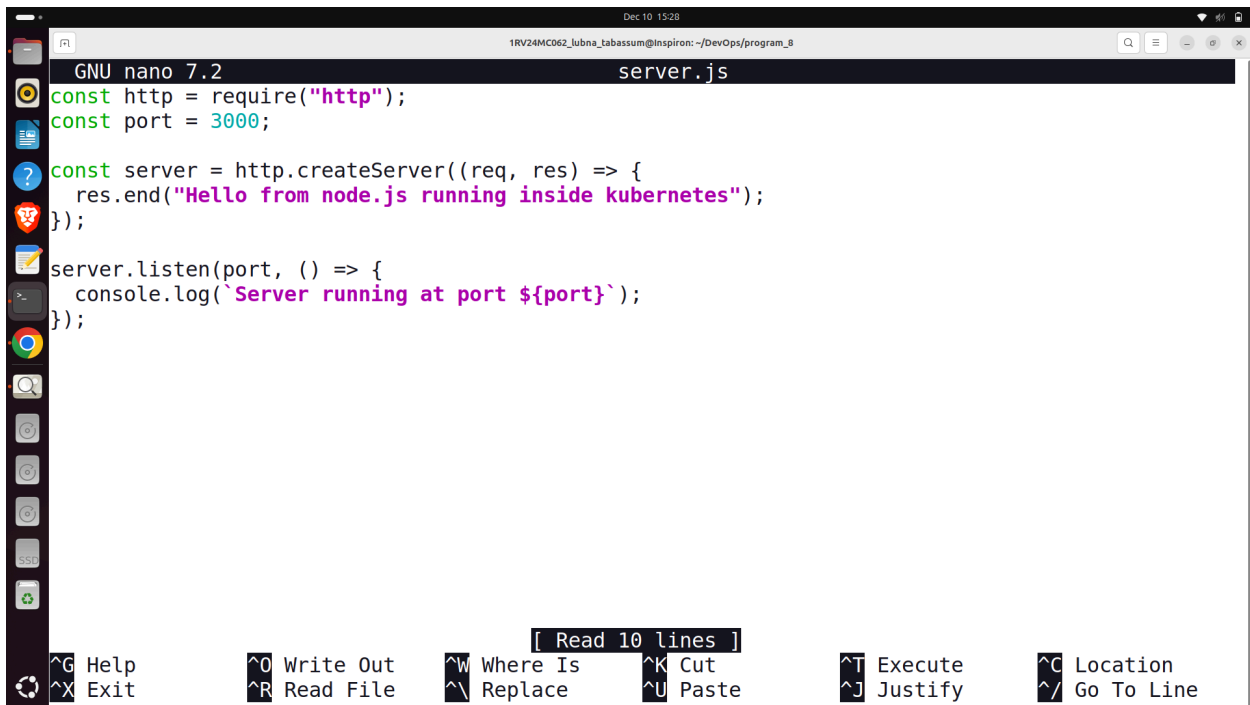


```
GNU nano 7.2 nodejs-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: program-8
  labels:
    app: nodejs
spec:
  containers:
    - name: nodejs
      image: node:18
      command: ["node", "/usr/src/app/server.js"]
      volumeMounts:
        - name: app-code
          mountPath: /usr/src/app
      ports:
        - containerPort: 3000
  volumes:
    - name: app-code
      configMap:
        name: nodejs-app-config
```

[Read 20 lines]

Help Write Out Where Is Cut Execute Location
Exit Read File Replace Paste Justify Go To Line

Step 4: Create `server.js` file with following code
`nano server.js`



```
GNU nano 7.2 server.js
const http = require("http");
const port = 3000;

const server = http.createServer((req, res) => {
  res.end("Hello from node.js running inside kubernetes");
});

server.listen(port, () => {
  console.log(`Server running at port ${port}`);
});
```

[Read 10 lines]

Help Write Out Where Is Cut Execute Location
Exit Read File Replace Paste Justify Go To Line

Step 5: Apply the yaml files

`docker apply -f nodejs-configmap.yaml`

```
1RV24MC062_lubna_tabassum@Inspiron:~/DevOps/program_8$ kubectl apply -f nodejs-configmap.yaml
configmap/nodejs-app-config unchanged
```

`docker apply -f nodejs-pod.yaml`

```
configmap/nodejs-app-config unchanged
1RV24MC062_lubna_tabassum@Inspiron:~/DevOps/program_8$ kubectl apply -f nodejs-pod.yaml
pod/program-8 created
```

Step 6: Get the pods

`docker get pods`

```
1RV24MC062_lubna_tabassum@Inspiron:~/DevOps/program_8$ kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
program-8     1/1     Running   0           9s
```

Step 7: Access the nodejs application

`kubectl port-forward pod/program-8 3000:3000`

```
1RV24MC062_lubna_tabassum@Inspiron:~/DevOps/program_8$ kubectl port-forward pod/program-8 3000:3000
Forwarding from 127.0.0.1:3000 -> 3000
Forwarding from [::1]:3000 -> 3000
Handling connection for 3000
Handling connection for 3000
```

Step 8: Access on the browser

<http://localhost:3000>

Output



Hello from Node.js running inside Kubernetes!